

Dev Interview Simulator

Guide de Référence : Setup Environnement Fullstack (Spring Boot 3 & React 19)

1. BACKEND (Spring Boot 3 + PostgreSQL 18 + Docker)

1.1. Squelette du Projet & Dépendances Maven

Généré via Spring Initializr. Fichier principal de configuration : `pom.xml` .

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.3</version>
        <relativePath/>
    </parent>

    <groupId>com.interview.simulator</groupId>
    <artifactId>backend</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>backend</name>
    <description>Dev Interview Simulator Backend</description>

    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

1.2. Configuration de l'Application (`application.properties`)

```
spring.application.name=backend
spring.datasource.url=jdbc:postgresql://localhost:5432/dev_interview_db
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

1.3. API Health Check Controller (`HealthController.java`)

```
package com.interview.simulator;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class HealthController {

    @GetMapping("/health")
    public Map<String, String> healthCheck() {
        return Map.of(
            "status", "UP",
            "service", "dev-interview-simulator-backend"
        );
    }
}
```

1.4. Multi-Stage Dockerfile

```
# Étape 1 : Build
FROM maven:3.9.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Étape 2 : Runtime
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

1.5. Orchestration avec Docker Compose (`docker-compose.yml`)

```
services:
  postgres:
    image: postgres:18-alpine
    container_name: dev-interview-postgres
    environment:
      POSTGRES_DB: dev_interview_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  backend:
    build: .
    container_name: dev-interview-backend
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/dev_interview_db
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: postgres
    depends_on:
      postgres:
        condition: service_healthy
```

Commandes Backend clés :

```
# Validation compilation & tests local
mvn clean compile
mvn test

# Lancement de la stack Docker
docker compose up --build -d
curl http://localhost:8080/api/health
```

2. FRONTEND (React 19 + TypeScript + Vite + Tailwind CSS 4)

2.1. Création du Projet Vite

```
npm create vite@latest dev-interview-simulator-frontend -- --template react-ts
cd dev-interview-simulator-frontend
```

2.2. Installation & Integration Tailwind CSS v4

```
npm install tailwindcss @tailwindcss/vite
```

Configuration de `vite.config.ts` :

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
})
```

Inclusion CSS dans `src/index.css` :

```
@import "tailwindcss";
```

2.3. Composant Validation UI Mobile-First (`src/App.tsx`)

```
import { useState } from 'react';

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <main className="min-h-screen bg-slate-900 text-slate-100 flex items-center justify-center p-4">
      <div className="w-full max-w-md md:max-w-xl lg:max-w-2xl bg-slate-800 border border-slate-700 rounded-2xl p-6 sm:p-8 shadow-xl text-center space-y-6">
        <h1 className="text-2xl sm:text-3xl md:text-4xl font-bold tracking-tight text-emerald-400">
          Dev Interview Simulator
        </h1>
        <p className="text-sm sm:text-base text-slate-300">
          Environnement Frontend (React 19 + TypeScript + Vite + Tailwind CSS 4) validé.
        </p>

        <div className="pt-2">
          <button
            onClick={() => setCount((c) => c + 1)}
            className="w-full sm:w-auto px-6 py-3 bg-emerald-500 hover:bg-emerald-600 active:bg-emerald-700 text-slate-950 font-semibold rounded-lg transition-colors cursor-pointer"
          >
            Compteur de test : {count}
          </button>
        </div>
      </div>
    </main>
  );
}
```

Commandes Frontend clés :

```
# Démarrage serveur dev  
npm run dev
```

```
# Build de production (tsc + vite build)  
npm run build
```